The server-side language supported by Razor Pages is C#, while the view templates are mostly composed of the languages of the web—HTML, CSS, and JavaScript. The placeholders for the dynamic content discussed earlier are C# code. You use Razor, a simple and easy-to-learn templating syntax, to embed server-side expressions and code in the view template. You don't need to learn any new languages to use Razor Pages. You don't even really need to know SQL to access a database because .NET includes frameworks that you will use to generate that for you.

1.2 What can you do with Razor Pages?

Razor Pages is a page-focused framework. Its primary purpose is to generate HTML. Therefore, it is suitable for creating *any* web application or any part of a web-based application that consists of web pages. In fact, it's probably easier to list the things you can't do with Razor Pages!

You looked at Manning's website earlier—an online catalog and e-commerce site. I'm reliably informed that it hasn't been built with Razor Pages, but it could have been. I've used Razor Pages for my blog and tutorial sites, where data is stored in databases or as Markdown files that need to be converted to HTML. I have also used Razor Pages in my day job for building magazine websites, internal web-based tools for managing business-related workflows and reporting, and even a custom content management system. Any kind of web application that includes pages as part of the requirement is a candidate for Razor Pages—from a simple blog site to the next eBay.

Razor Pages is particularly suitable for any type of forms-based application. Often referred to as CRUD applications, create, read, update, and delete represent the four basic operations related to persistent storage of a model.